

Multithreading

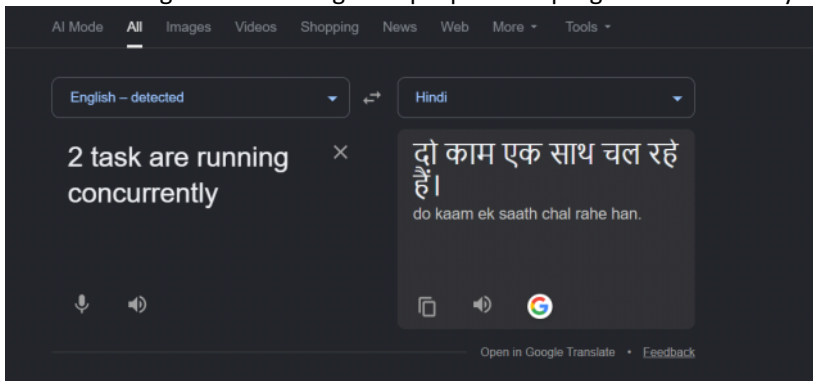
11 July 2026 09:05

You already know:

1. Inheritance
2. Interface
3. Exception handling
4. Custom exception - 16 page homework
5. Abstract classes

Main idea:

Multithreading means running multiple part of a program concurrently



One program can do many tasks.

One task can run in one thread.

Examples:

Youtube:

- play video
- load comment
- show ads
- track like

Food delivery app:

- search restaurants
- track delivery partner
- Apply coupon
- process payment

Before starting anything:

Can a person listen to music and write notes at the same time.

Ans: yes both task can happen together.

Similarly, a program can perform multiple task together using **threads**.

Multitasking:

1. **Process based multitasking** : executing task simultaneously where each task is separate independent process.
2. **Thread based multithreading** : executing task simultaneously where each task is separate independent part of program.

What is Thread ?

A **thread** in Java is the smallest unit of execution within a program that allows tasks to run concurrently.

Definition:

Thread is piece of code running Independent but part of program.

Thread = lightweight sub-task of program.

Example:

Program : VLC player

Thread 1: play song

Thread 2 : show timer

Thread 3: handle play/pause

Note:

Thread is not separate program, thread is part of a program.

Process And Thread

Process:

1. running program
2. separate memory
3. Slower
4. Chrome application

Thread:

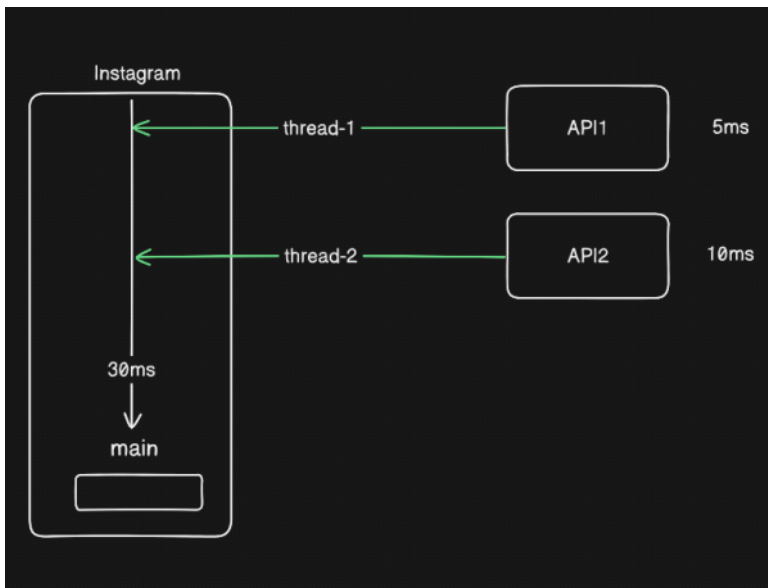
1. small task inside process
2. Share memory of process
3. Fast
4. One tab loading, one file downloading

Note:

A process can have multiple threads, thread inside same program share memory.

Why we need multithreading ?

1. Better performance
2. Running independent task together



Important:

1. Concurrent does not always mean exactly at the same millisecond or same time.

Sequential program:

How to create thread:

1. By extending thread class
2. By Implementing Runnable interface
3. Using lambda expression
4. Using ExecutorService

```

main 0
Thread-0 0
main 1
Thread-0 1
main 2
Thread-0 2
main 3
Thread-0 3
main 4
Thread-0 4
Process finished with exit code 0

```

Why this random output ?

Thread Scheduler:

When multiple thread are waiting for execution, which thread will get chance to execute is make by "Thread Scheduler". It is an internal component of JVM. The specific algorithm or behaviour can vary depending on JVM vendor.

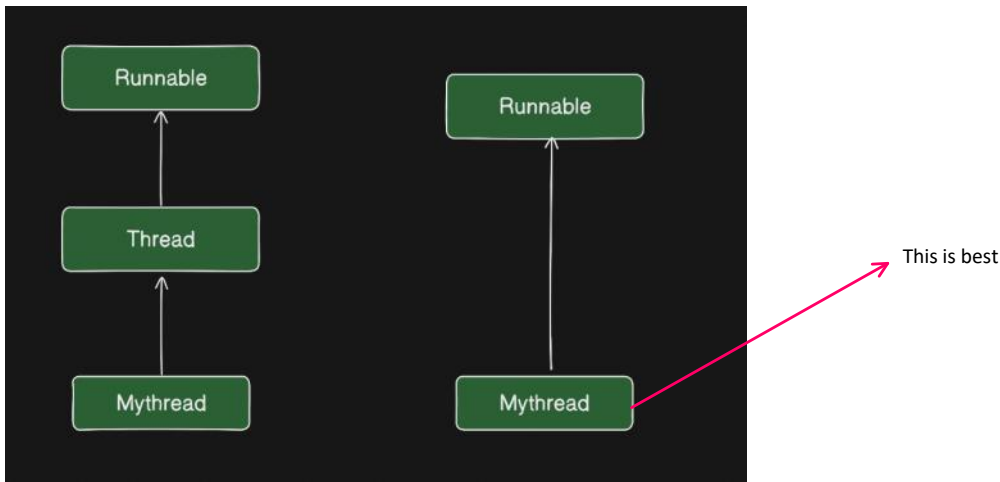
Difference between t.start() and t.run()

When we use t.start() a new thread is created and this new thread is responsible for executing run() method
 When we use t.run() no new thread will be created run() method will be executed like regular method.

Important:

Creating a task is responsibility of run() which is from runnable.

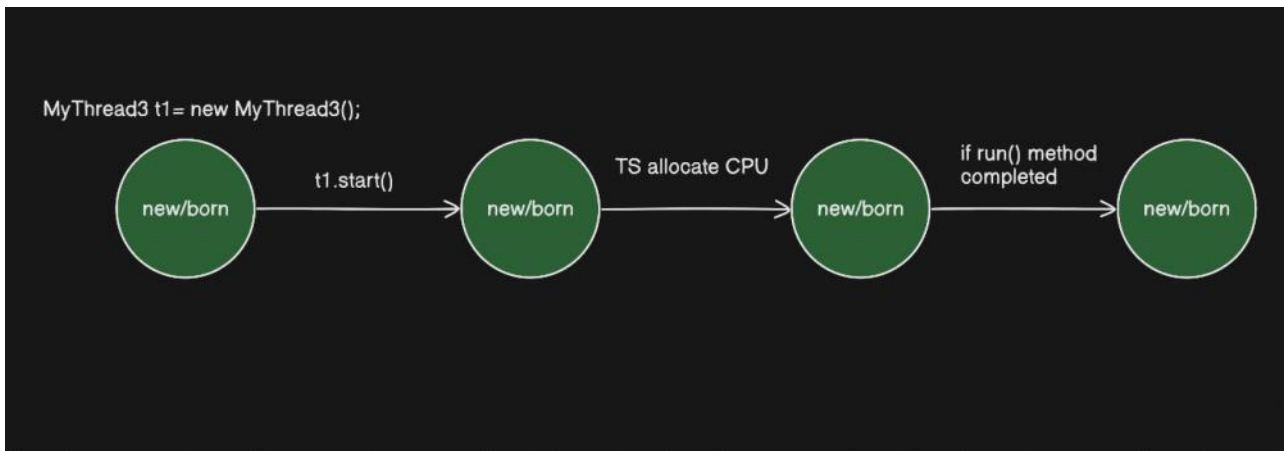
Create a thread is responsibility of thread class because start() method is only available in Thread class



```
3 class MyThread3 extends Thread 2 usages
4 {
5     @Override
6     public void run() {
7         System.out.println(Thread.currentThread().getName()+ " child");
8         System.out.println("Running...");
9     }
10 }
11 public class Test {
12     public static void main(String[] args) {
13
14         System.out.println(Thread.currentThread().getName()+"main ");
15         MyThread3 t1= new MyThread3();
16         t1.start();
17         t1.start();
18         //on thread object can be start only once
19     }
20 }
```

```
mainmain
Thread-0 child
Running...
Exception in thread "main" java.lang.IllegalThreadStateException Create breakpoint
    at java.base/java.lang.Thread.start(Thread.java:1416)
    at Test.main(Test.java:18)
Process finished with exit code 1
```

Life cycle of thread



Thread Priorities

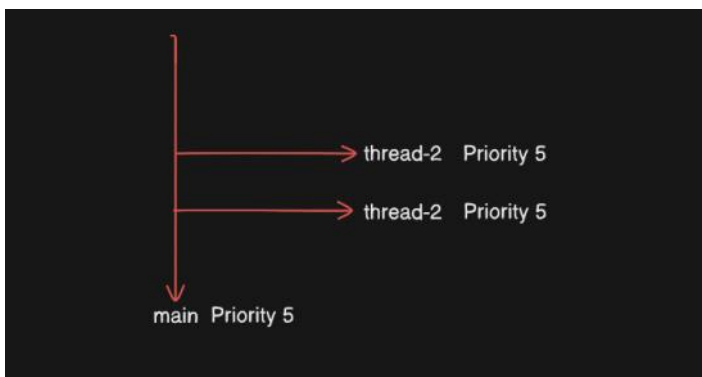
In java thread can have different priorities that indicate their importance to the thread scheduler.

Priorities can be from 1 to 10

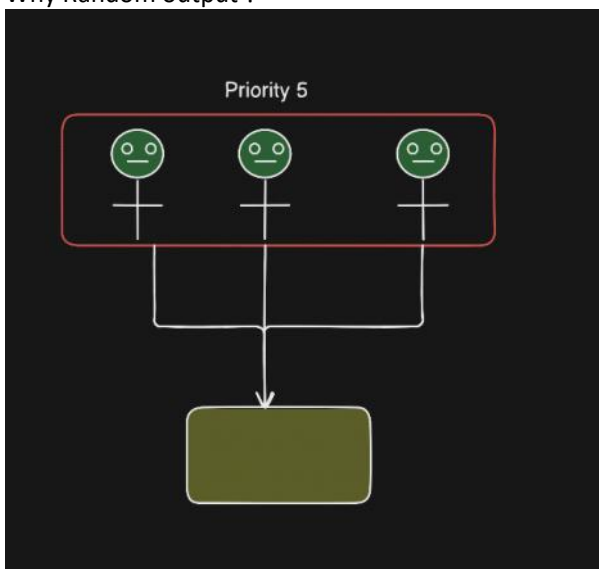
1 -- lowest priority

5 - Normal priority

10 -- highest priority



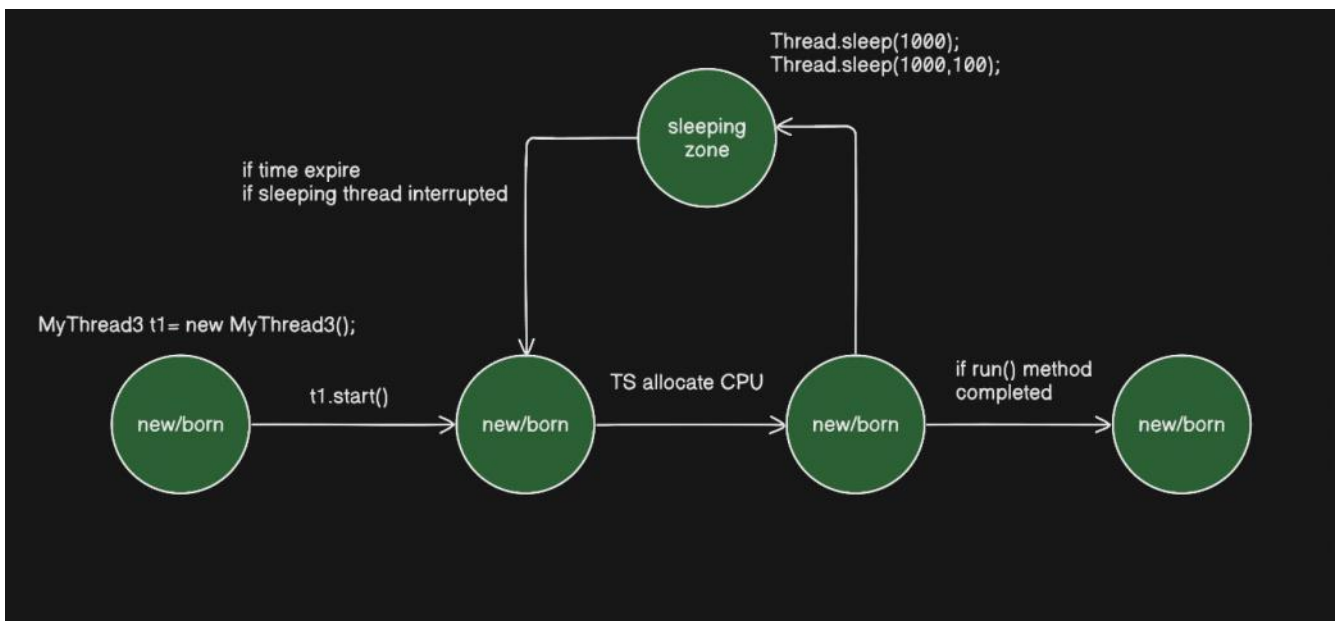
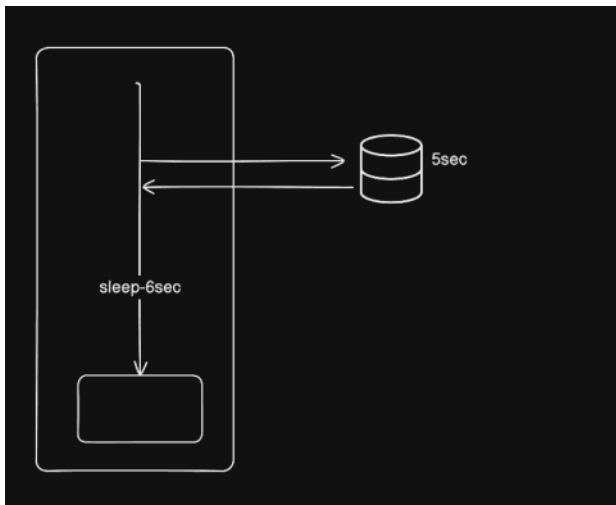
Why Random output ?



The Method to prevent a thread from execution:

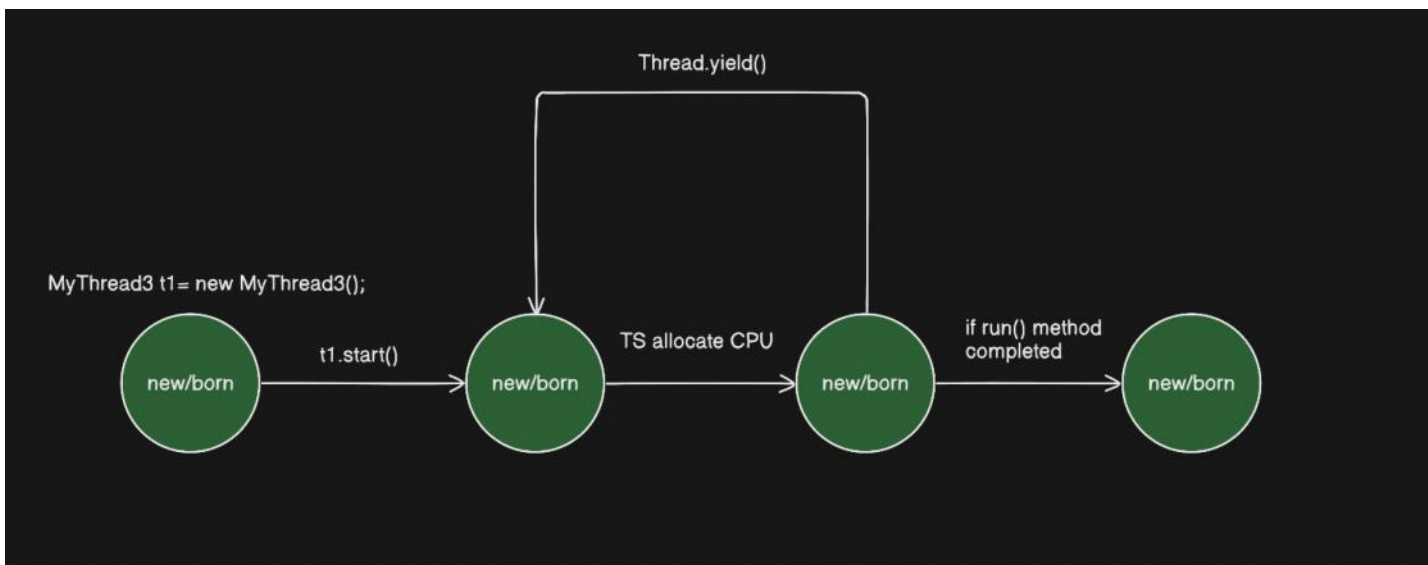
1. Yield()
2. Join()
3. Sleep()

Sleep() pause current thread for some time:



Thread.yield()

This method giving hint to the scheduler that the current thread is willing to yield it's current use of the processor. It allow other thread of equal priority to rub before the current thread resume execution.



```

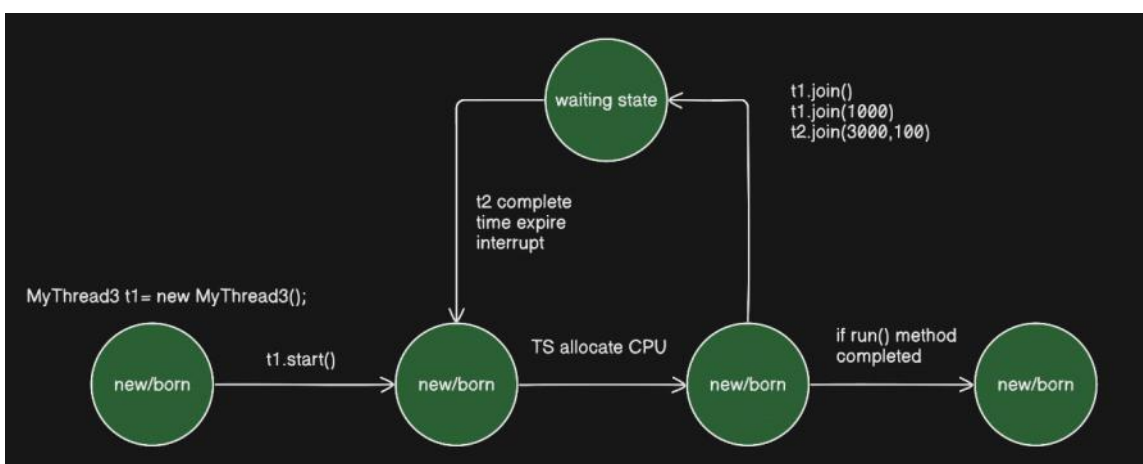
2
3 public class YieldExample {
4     public static void main(String[] args) {
5         MyThread5 t1= new MyThread5();
6         t1.start();
7         for (int i = 0; i < 5 ; i++) {
8             System.out.println("Main thread");
9         }
10    }
11
12    class MyThread5 extends Thread 2 usages
13    {
14        @Override
15        public void run() {
16            Thread.yield();    ///line 17 -->child
17            for (int i=0;i<10;i++)
18            {
19                System.out.println(Thread.currentThread().getName()+" : "+i);
20            }
21        }
22    }
  
```

Note:

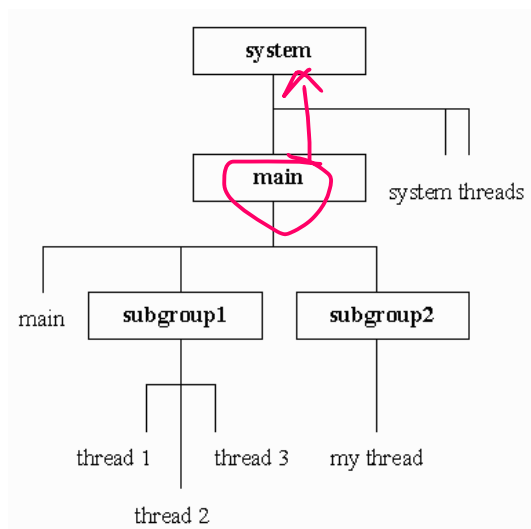
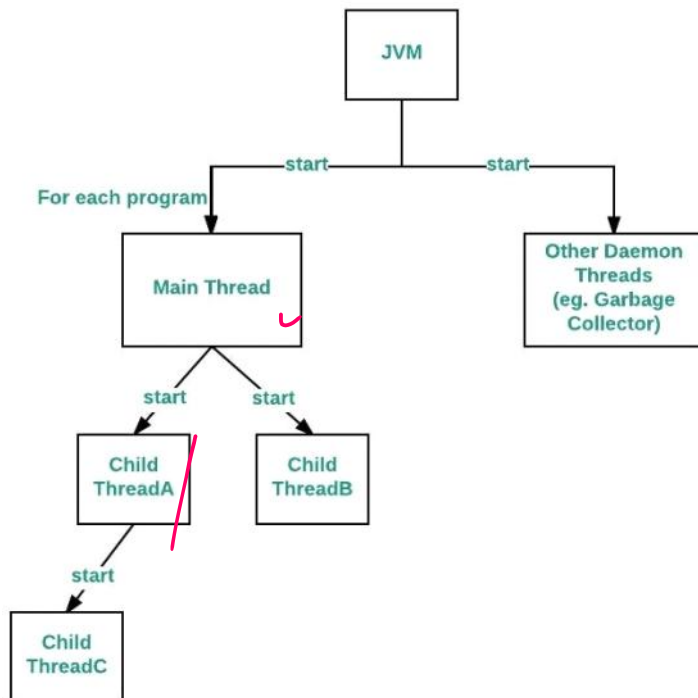
In above program child thread calling yield() method and hence main thread will get chance more number of time for execution

Hence the chance of completing the main thread is first is high.

Join()

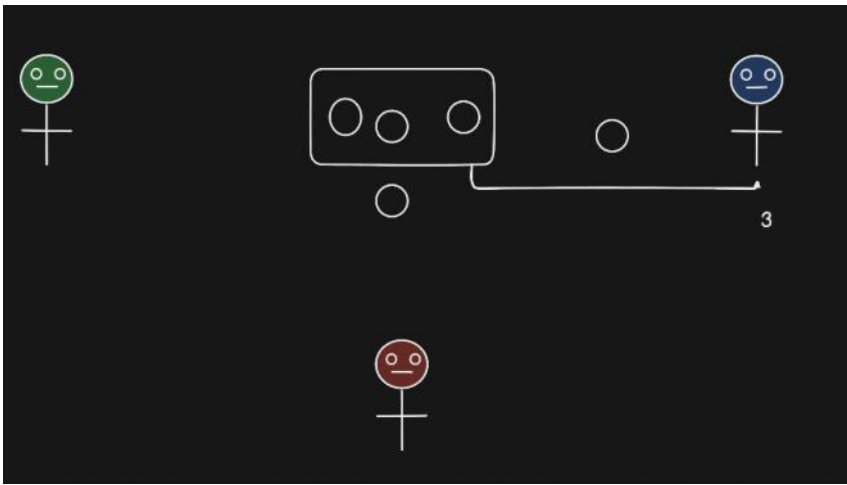


Join() makes one thread wait until another thread completes.



Race condition

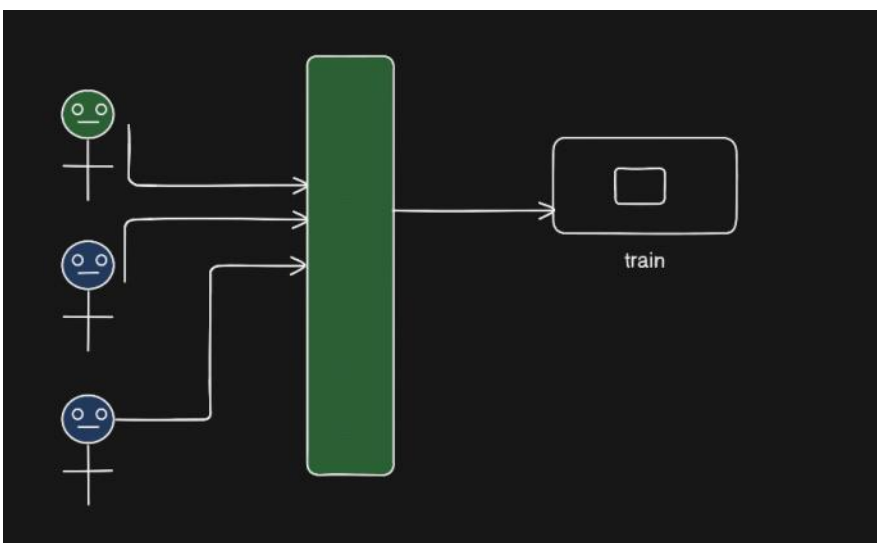
Race condition happens when multiple thread access shared data and final result depends on thread timing



When this type of problem occurs:

1. Two user booking
2. Multiple users update

Synchronization



Allow only one thread at a time to access a critical section.

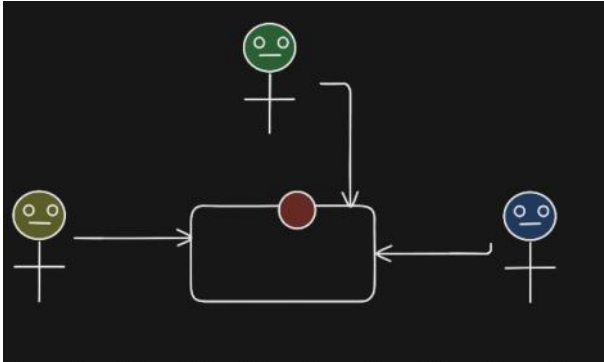
1. Synchronized keyword is applicable for method and block but not for classes
2. When we make method or block Synchronized only one thread is allowed to execute that method or block at a time on the given object
3. Primary advantage of Synchronized is to resolve data inconsistency
4. However major drawback of Synchronized is that it will increase waiting time of threads and negatively the performance of the system will be decrease.
5. Therefore it is not recommended to use the Synchronized keyword unless there is specific requirement
6. Internally the Synchronized concept is implemented using locks
7. Each object in java has unique lock. The usage of the Synchronized keyword activates the lock concept
8. If thread want to execute Synchronized method it must be acquire lock on the object first.

Inter thread communication

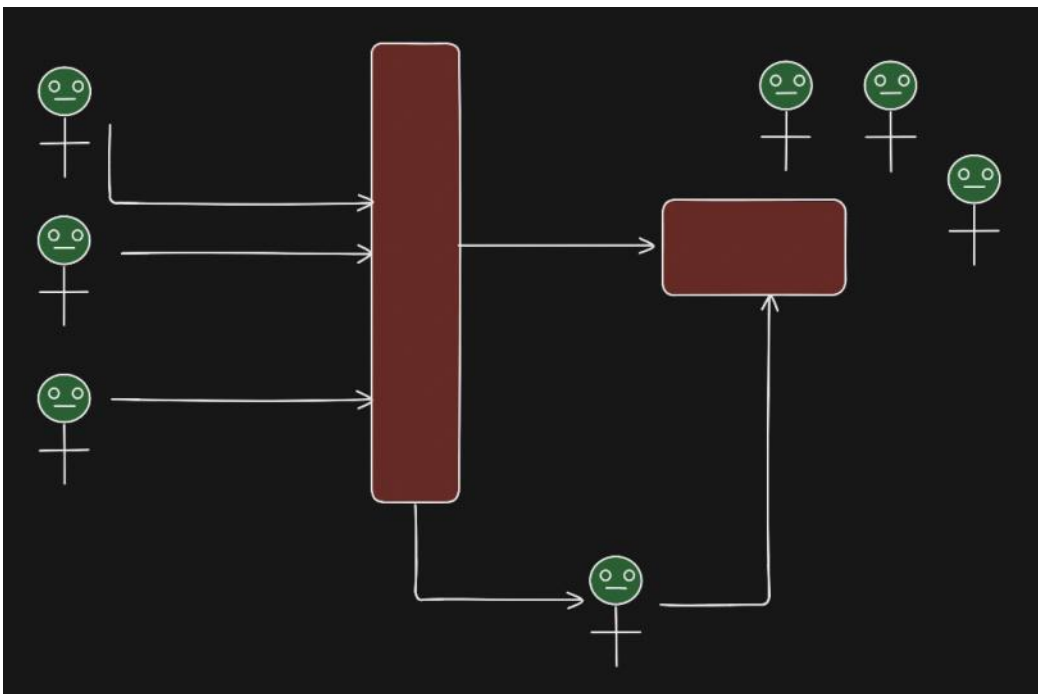
1. wait() :
2. notify()
3. notifyAll()

Inter-thread communication in java happen through these methods.

1. **Wait()**: this method is called on an object within **Synchronized** context to make the current thread wait until another thread invoke notify() or notifyAll() method on same object.
 2. It release the lock on the same object and enter into waiting sate until it receives a notification
-
1. **Notify()**: this method is called on an object within Synchronized context to wake up a single thread that is waiting on the same object.
 2. When notify() in invoked notifies one of the thread that are waiting on the object to wake up. But which thread will get chance it depends on TS.
-
1. **notifyAll()**: this method is called on an object within Synchronized context to wake up all thread that is waiting on the same object.



Join me aap thread ka wait krte ho
Wait me aap lock ka wait kr rahe ho

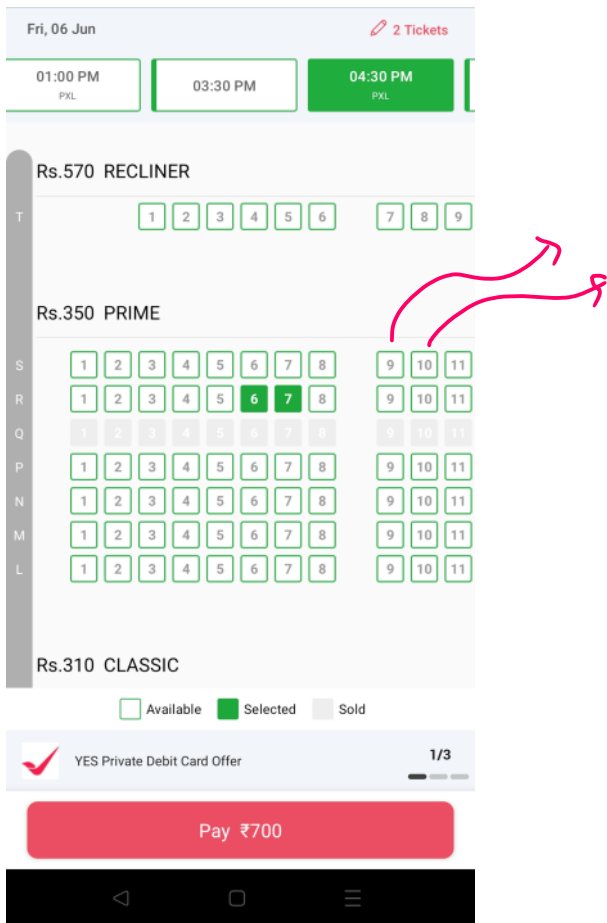


Lock Farmwork and ReentrantLock

Lock is like synchronized, but with more control

Synchronized = automatic locking

Lock = manual locking



Executor Framework

Till now we created thread manually. That is good for learning but in real projects, we usually do not create a new thread for every task

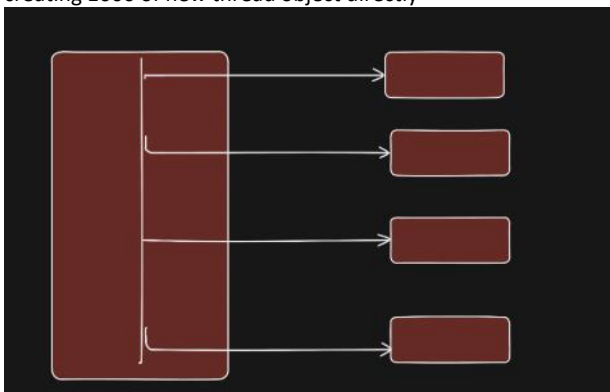
We use Executor Framework

Real problem:

Suppose 10000 student register for a webinar.
After reg, application must send 1000 confirmation emails

Bad approach:

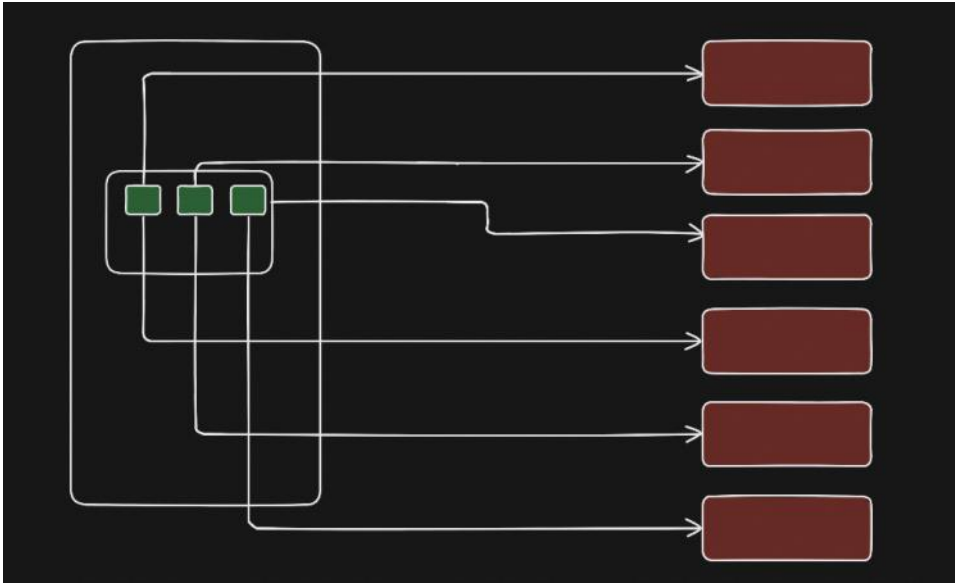
creating 1000 of new thread object directly



Better approach

Create a fixed size thread pool like 5,10,14

Give all email task to that pool, the thread you will create inside it act as worker



Executor farmwork = readymade thread management system

Callable & Future

Runnable does not return value, callable can return value.

3hr + 3.5 == 6.5

20hr : 1.0 + 1.5 :

sir devops aur cloud toh yeha pe bhi cover hoga na i mean bit off and what will be extra in that course the new one